

INTRODUCTION ASP.NET– MVC CORE

Cours présenté par Tarek Ayari

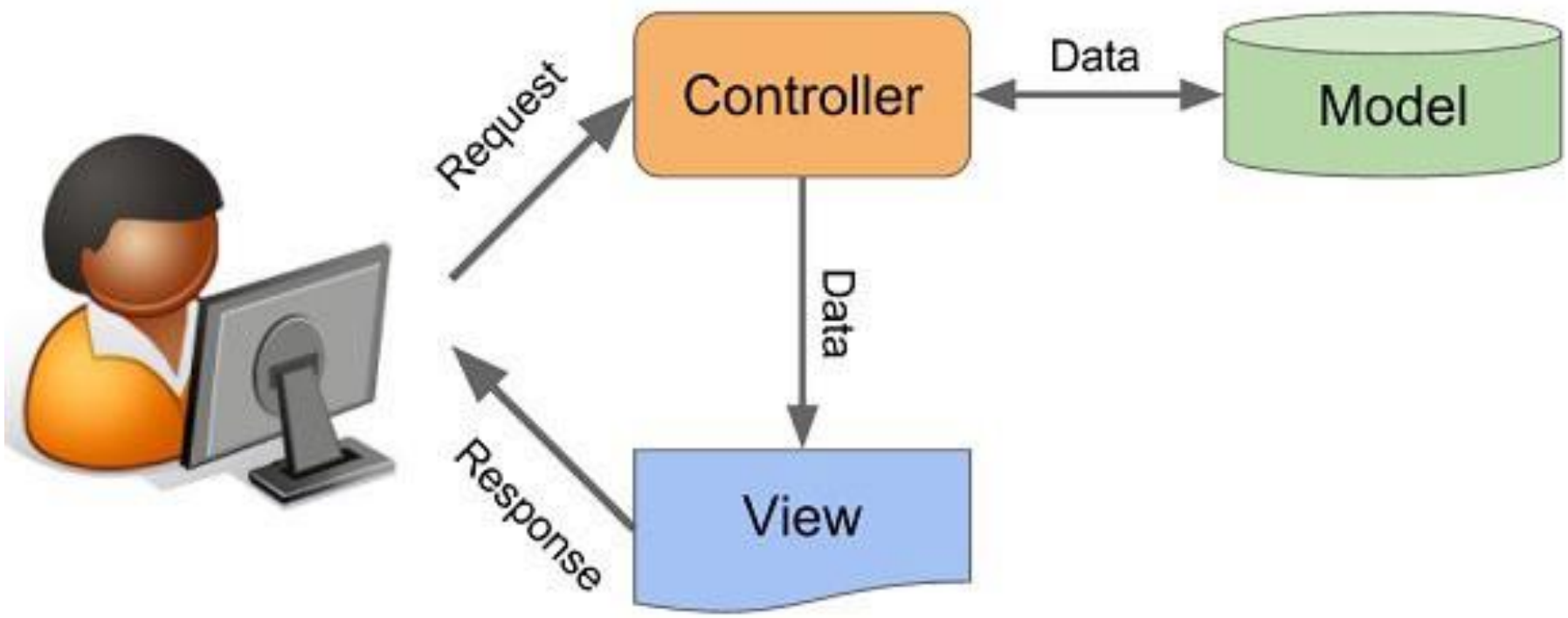
PLAN DU COURS

- Le modèle MVC
- ARCHITECTURE D'UNE APPLICATION MVC
- Rôles des composants
- Interactions entre les composants
- Avantages du Modèle MVC

LE MODÈLE MVC

- Le modèle MVC décrit une manière d'architecturer une application informatique en la décomposant en trois sous-parties :
- la partie Modèle ;
- la partie Vue ;
- la partie Contrôleur.
- Ce modèle de conception (design pattern) a été imaginé à la fin des années 1970 pour le langage Smalltalk afin de bien séparer le code de l'interface graphique de la logique applicative. Il est utilisé dans de très nombreux langages : bibliothèques Swing et Model 2 (JSP) de Java, frameworks PHP, ASP.NET MVC, etc.

ARCHITECTURE D'UNE APPLICATION MVC



RÔLES DES COMPOSANTS : MVC – M

- Le modèle représente le cœur de l'application. Il est la partie qui va manipuler les données et/ou information d'une Base de données. Il permet de mettre à jour ces données comme par exemple les supprimer, modifier leur valeur etc…

- La partie Modèle d'une architecture MVC encapsule la logique métier (business logic) ainsi que l'accès aux données. Il peut s'agir d'un ensemble de fonctions (Modèle procédural) ou de classes (Modèle orienté objet).

RÔLES DES COMPOSANTS : MVC – V

- La vue est toute la partie graphique de notre application. Son but premier est d'afficher les données du Modèle. Mais elle est également celle qui va recevoir tous les évènements que vas produire l'utilisateur (Clic de souris, mouvement, case cochée etc).

- Plusieurs vues peuvent afficher différentes informations du même modèle.

RÔLES DES COMPOSANTS : MVC – C

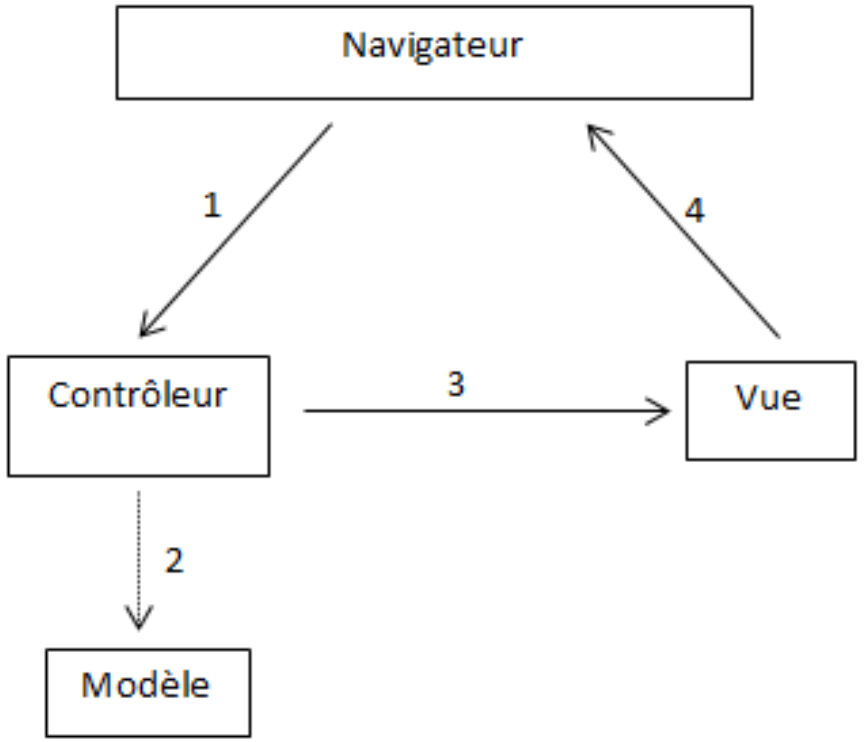
- Le contrôleur va, en fonctions des évènements produits par l'utilisateur, mettre à jour la vue ou le modèle et synchroniser ceux-ci.

- Il va, en fonction des actions de l'utilisateur effectuer diverses actions. Si, par les évènements de l'utilisateur, la vue doit être modifiée, le contrôleur va lui signaler. Inversement, si des données doivent être modifiées le contrôleur vas le signaler au modèle qui va s'en charger.

- La partie Contrôleur gère la dynamique de l'application. Elle fait le lien entre l'utilisateur et le reste de l'application

INTERACTIONS ENTRE LES COMPOSANTS

- Voici un petit schéma synthétique qui explique comment les différentes couches interagissent entre elles :



1. Action utilisateur via une requête HTTP
2. Consultation et/ou mise à jour du modèle (facultatif)
3. Le contrôleur décide de la vue à afficher
4. La vue renvoi le HTML au navigateur

AVANTAGES DU MODÈLE MVC

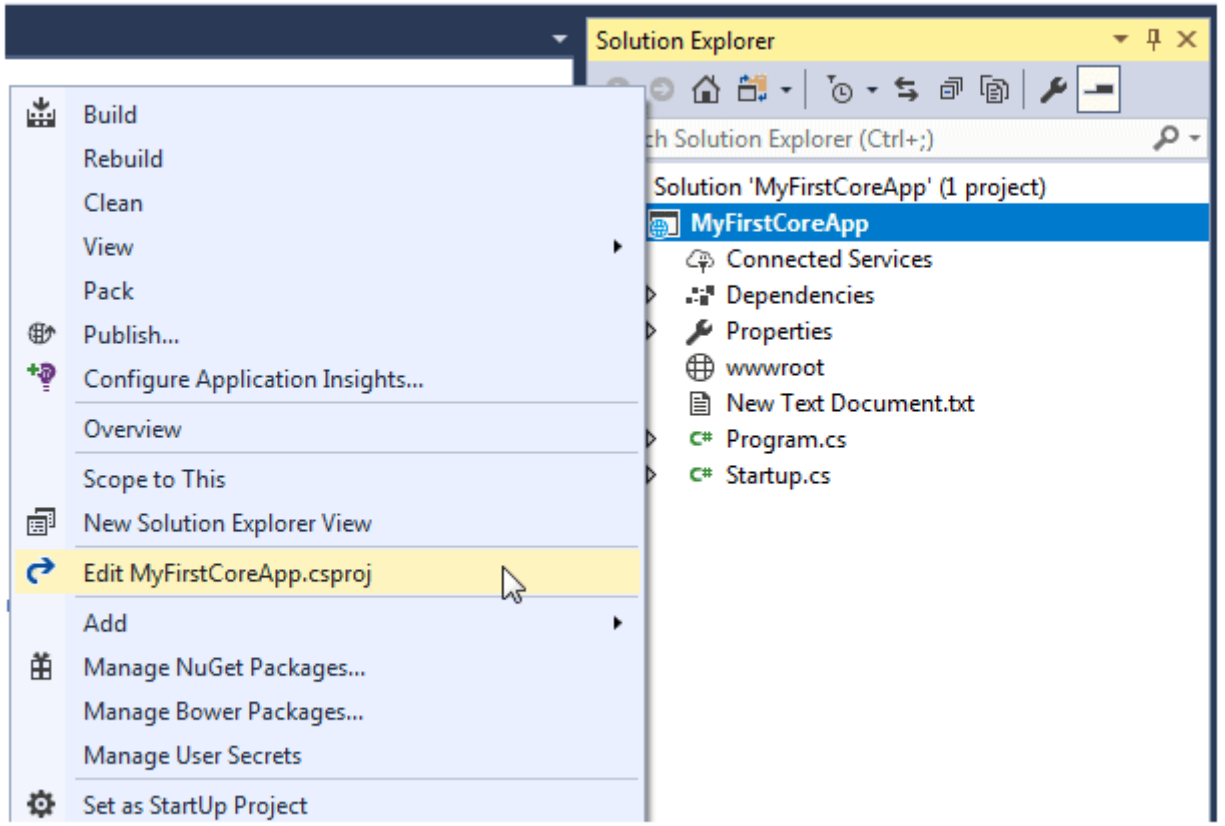
- Le modèle MVC offre une séparation claire des responsabilités au sein d'une application, en conformité avec les principes de conception déjà étudiés : responsabilité unique, couplage faible et cohésion forte. Le prix à payer est une augmentation de la complexité de l'architecture.

- Dans le cas d'une application Web, l'utilisation du modèle MVC permet aux pages HTML (qui constituent la partie Vue) de contenir le moins possible de code serveur, étant donné que le scripting est regroupé dans les deux autres parties de l'application

ARCHITECTURE PROJET ASP.NET MVC CORE

1/ .csproj

Visual Studio utilise désormais le fichier .csproj pour gérer les projets.
Nous pouvons modifier les paramètres .csproj en cliquant avec le bouton droit sur le projet et en sélectionnant Edit <project-name>.csproj comme indiqué ci-dessous.



Modifier .csproj

ARCHITECTURE PROJET ASP.NET MVC CORE

1/ .csproj

Le fichier .csproj pour le projet ci-dessus ressemble à ci-dessous. Le fichier csproj comprend des paramètres liés aux frameworks .NET Core ciblés, aux dossiers de projet, aux références de package NuGet

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>netcoreapp2.1</TargetFramework>
  </PropertyGroup>

  <ItemGroup>
    <Folder Include="wwwroot\" />
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.App" />
    <PackageReference Include="Microsoft.AspNetCore.Razor.Design" Version="2.1.2" PrivateAssets="All" />
  </ItemGroup>

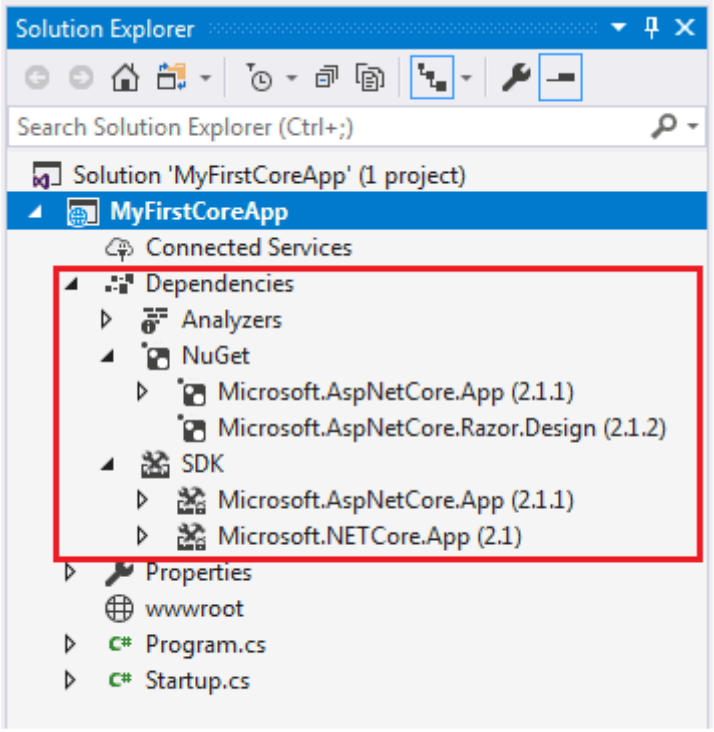
</Project>

```

ARCHITECTURE PROJET ASP.NET MVC CORE

2/ Dépendances

Les dépendances du projet ASP.NET Core contiennent tous les packages NuGet installés côté serveur, comme indiqué ci-dessous.

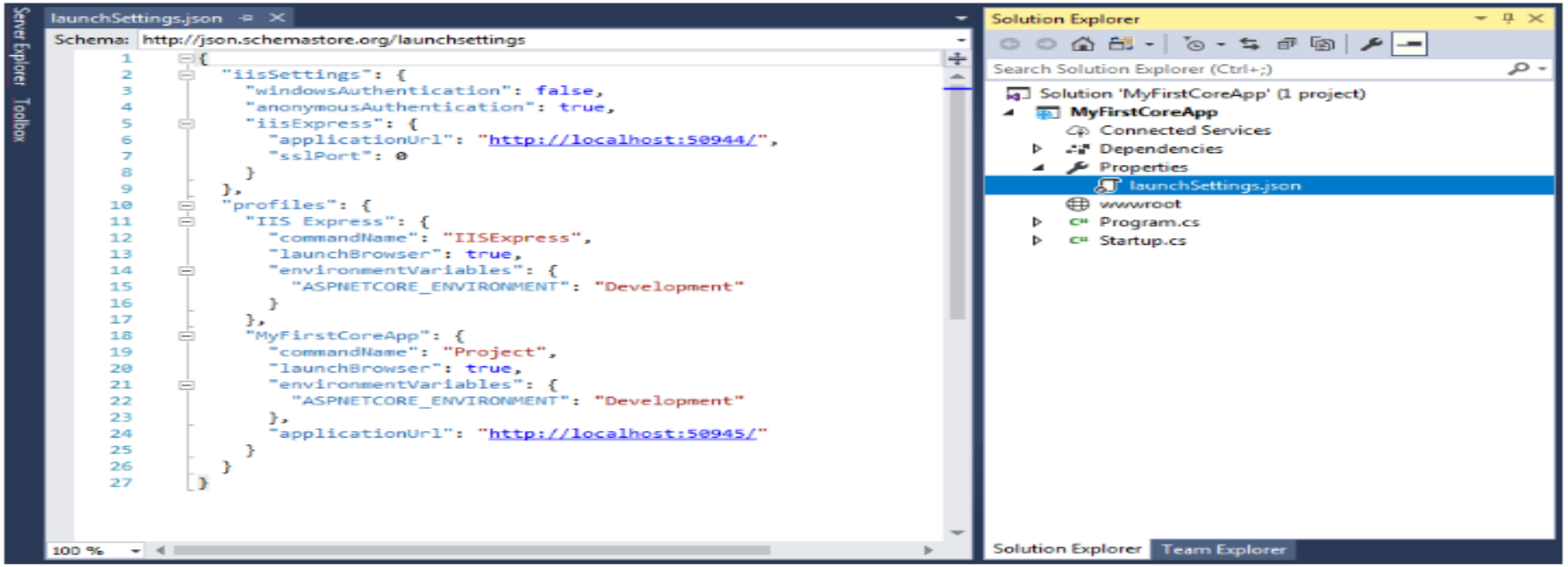


Dépendances

ARCHITECTURE PROJET ASP.NET MVC CORE

3/ Propriétés

Le nœud Propriétés inclut le fichier launchSettings.json qui inclut les profils Visual Studio des paramètres de débogage..

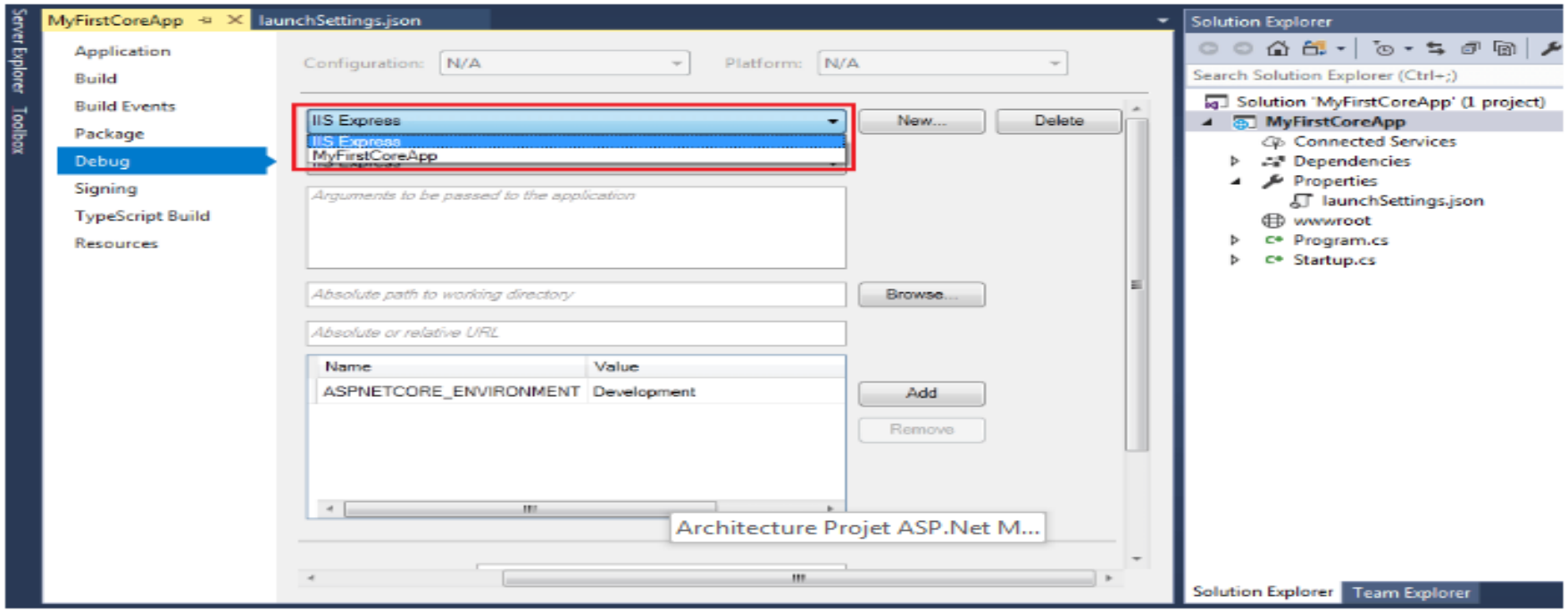


launchSettings.json

ARCHITECTURE PROJET ASP.NET MVC CORE

3/ Propriétés

Nous pouvons également modifier les paramètres à partir de l'onglet de débogage des propriétés du projet. Faites un clic droit sur le projet -> sélectionnez Propriétés
-> cliquez sur l'onglet Déboguer



Propriétés du projet

ARCHITECTURE PROJET ASP.NET MVC CORE

4/ Dossier wwwroot

Par défaut, le dossier **wwwroot** du projet ASP.NET Core est traité comme un dossier racine Web.

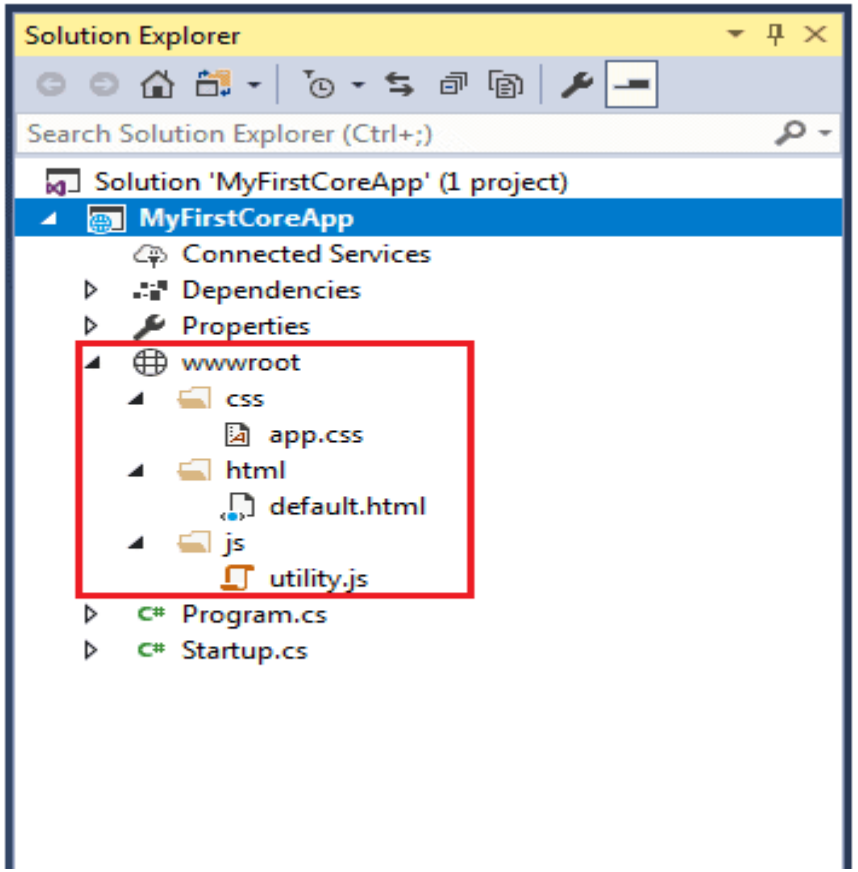
Les fichiers statiques peuvent être stockés dans n'importe quel dossier sous la racine Web et accessibles avec un chemin relatif vers cette racine.

il devrait y avoir des dossiers séparés pour les différents types de fichiers statiques tels que JavaScript, CSS, Images, scripts de bibliothèque, etc

Vous pouvez accéder à des fichiers statiques avec une URL de base et un nom de fichier. Par exemple, nous pouvons accéder au fichier app.css ci-dessus dans le dossier css par http://localhost:<port>/css/app.css.

ARCHITECTURE PROJET ASP.NET MVC CORE

4/ Dossier wwwroot



ARCHITECTURE PROJET ASP.NET MVC CORE

5/ appsettings.json

Le fichier appsettings.json stocke les valeurs de configuration dans des paires nom-valeur au format JSON.

Le système de configuration ASP.NET Core charge le fichier appsettings.json qui contient les paramètres de configuration qui vont être utilise par l'application.

```
1
2  {
3    "Logging": {
4      "IncludeScopes": false,
5      "LogLevel": {
6
7        "Default": "Warning"
8      }
9    },
10
11 "AllowedHosts": "*"
12 ,
13
14   "ConnectionStrings": {
15     "DefaultConnection":"Server=LocalHost;Database=TestDB;Integrated Security=false;uid
16   },
17
```